



CSC 316–Lecture Seven

Regular Expressions

Lecture Outlines

- Introduction
- What is a Language?
- Introduction to Regular Expressions (RegExps)
- Regular Definitions and Notations
- Derivations, Parenthesis and Precedence Rules
- Use of RegExps in a Typical Programming Language
- RegExp and NFA

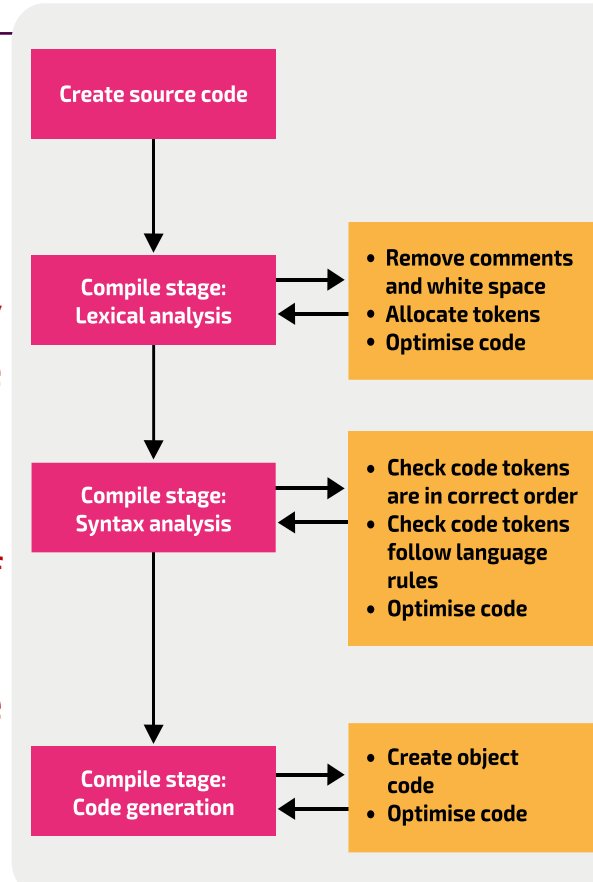
Recap: Stages of Compilation

1. Lexical Analysis

- Comments and unnecessary spaces are removed.
- Keywords, constants and identifiers are replaced by 'tokens', which are symbolic strings to identify what the elements are.

2. Symbol Table Construction

- A **table stores** the names and addresses of all variables, constants and arrays.
- Variables are checked to make sure they have been declared and to determine the data types used.



Recap: Stages of Compilation

3. Syntax Analysis

- Tokens are checked to see if they match the syntax of the programming language.
- If syntax errors are found, error messages are produced

4. Semantic Analysis

- Variables are checked to make sure they have been correctly declared and contain the correct data type.
- Operations are checked to ensure

that they are appropriate for the type of variable being used.

5. Code Generation

- Target Machine code is generated in this stage.

6. Code Optimisation

- Code optimisation makes the program more efficient so it runs faster and uses fewer resources.

Lecture Outlines | *Progress*

- **Introduction**

- What is a Language?
- Introduction to Regular Expressions (RegExps)
- Regular Definitions and Notations
- Derivations, Parenthesis and Precedence Rules
- Use of RegExps in a Typical Programming Language
- RegExp and NFA

Introduction

- The word “**lexical**” in the traditional sense means “pertaining to words”.
 - In terms of programming languages, words are objects like variable names, numbers, keywords etc.
 - Such words are traditionally called **Tokens**.
- The **Lexical Analyser** (lexer or scanner), takes a string as input of individual letters and divide this string into a sequence of classified ***tokens***.
 - Filter out whatever separates the tokens (the so-called white-space), i.e., lay-out characters (spaces, newlines etc.) and comments.

Introduction

- **Lexical Analysis** simplifies the **Syntax Analysis** phase.
 - Some reasons for keeping the phases separate include:
 - **Efficiency** - A specialised lexer may do the simple parts of the work faster than the parser
 - **Modularity** - The syntactical description of the language need not be cluttered with small lexical details such as white-space and comments;
 - **Tradition** - Languages are often designed with separate lexical and syntactical phases;

Introduction

- For lexical analysis, specifications are traditionally written using **Regular Expressions**.
 - **An algebraic notation for describing sets of strings.**
- The generated **lexers** are in a class of extremely simple programs called **Finite Automata**.

Lecture Outlines | *Progress*

- Introduction
- **What is a Language?**
- Introduction to Regular Expressions (RegExps)
- Regular Definitions and Notations
- Derivations, Parenthesis and Precedence Rules
- Use of RegExps in a Typical Programming Language
- RegExp and NFA

A Language

- The set of all integer constants or the set of all variable names are examples of sets of strings;
 - The individual **Digits** or **Letters** used to form these **constants** or **names** are taken from a particular **Alphabet** (i.e. a set of characters)
- A set of strings is called a **Language**.
 - For integers, the alphabet consists of the digits **0–9**
 - For variable names the alphabet contains both **Letters** and **Digits** (and in some cases, an **Underscore**).

Formalism

Definition (Formal Languages)

Let Σ be an *alphabet*, i.e., a finite set of allowed characters.

- **A word** over Σ is a string of chars $w = a_1 a_2 \dots a_n$, $a_i \in \Sigma$
 $n = 0$ is allowed and results in the empty string, denoted ϵ .
 Σ^* is the set of all words over Σ .
- **A language** L over Σ is a set of words over Σ , i.e., $L \subset \Sigma^*$.

Language Examples

- Number Literals in **C++**
 - **Integers in decimal format:** 234, 0, 8 (*but not **08** or **abc!***)
 - **Floating point decimals:** 0. or .345 or 123.45.
 - **Scientific notation:** 234E-45 or 0.E123 or .234e+45

Lecture Outlines | *Progress*

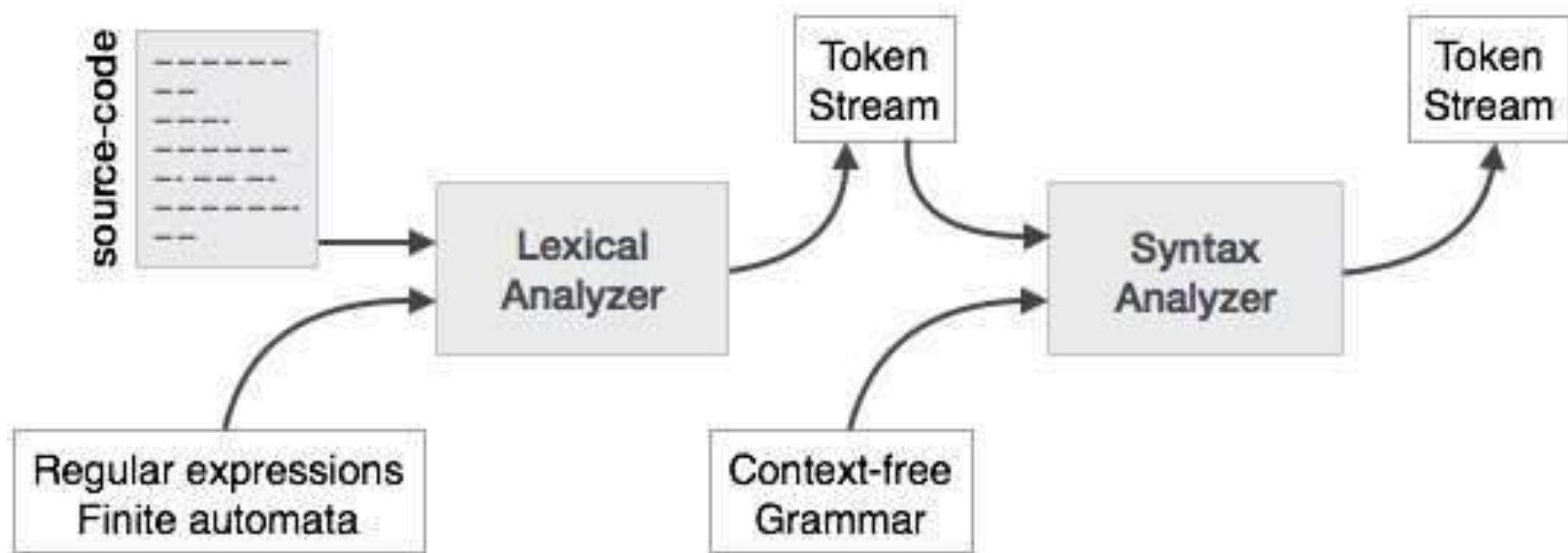
- Introduction
- What is a Language?
- **Introduction to Regular Expressions (RegExps)**
- Regular Definitions and Notations
- Derivations, Parenthesis and Precedence Rules
- Use of RegExps in a Typical Programming Language
- RegExp and NFA

Regular Expressions (Regexps)

- RegExps is an **Algebraic Notation** that is compact and relatively easy for humans to use and understand.
 - Provides a **Formal and Compositional** description of what tokens are.
 - RegExps are an automatic implementation of the token of a language.
- Given an **Alphabet**, all sets of strings of that alphabets can be defined by **Regular Expressions**.
 - **Regular Expressions** that describe simple sets of strings can be combined to form **regular expressions** to describe **more complex sets of strings**.

Role of RegExp in Compiler Design

- A notation to **represent lexeme patterns** for a **token**.
- Used to **represent the language** for **lexical analyzer**
- Assist in finding the type of **token** that accounts for a particular **lexeme**.
- They combine of **input symbols** and language operators such as **Union, Concatenation** and **Closure**.
 - **Union, Concatenation** and **Closure** are the **basic** operations on languages
- It can be used to describe the **identifier** for a language.



Lecture Outlines | *Progress*

- Introduction
- What is a Language?
- Introduction to Regular Expressions (RegExps)
- **Regular Definitions and Notations**
- Derivations, Parenthesis and Precedence Rules
- Use of RegExps in a Typical Programming Language
- RegExp and NFA

Formal Definition of RegExps

Definition (Regular Expressions)

The set $RE(\Sigma)$ of regular expressions over alphabet Σ is defined:

- *Base Rules (Non Recursive):*
 - $\epsilon \in RE(\Sigma)$ describes the lang consisting of *only the empty string*.
 - $a \in RE(\Sigma)$ for $a \in \Sigma$ describes the lang. of *one-letter word a*.
- *Recursive Rules: for every $\alpha, \beta \in RE(\Sigma)$*
 - $\alpha \cdot \beta \in RE(\Sigma)$, two language *sequence/concatenation* in which the first word is described by α , the second word by β .
 - $\alpha \mid \beta \in RE(\Sigma)$, *alternative/union*: lang described by α OR β .
 - $\alpha^* \in RE(\Sigma)$, *repetition*: zero or more words described by α .

Formal Definition of RegExps

Let us define regular expressions precisely:

A **regular expression** s is a string which denotes $L(s)$, a set of strings drawn from an alphabet Σ . $L(s)$ is known as the “language of s .”

$L(s)$ is defined inductively with the following base cases:

- If $a \in \Sigma$ then a is a regular expression and $L(a) = \{a\}$.
- ϵ is a regular expression and $L(\epsilon)$ contains only the empty string.

Then, for any regular expressions s and t :

1. $s|t$ is a RE such that $L(s|t) = L(s) \cup L(t)$.
2. st is a RE such that $L(st)$ contains all strings formed by the concatenation of a string in $L(s)$ followed by a string in $L(t)$.
3. s^* is a RE such that $L(s^*) = L(s)$ concatenated zero or more times.

Rule #3 is known as the **Kleene closure** and has the highest precedence. Rule #2 is known as **concatenation**. Rule #1 has the lowest precedence and is known as **alternation**. Parentheses can be added to adjust the order of operations in the usual way.

Examples of RegExps

- Here are a few examples using just the basic rules. (Note that a finite RE can indicate an infinite set.)
- | Regular Expression | Language |
|-------------------------|--|
| <code>L(s) hello</code> | <code>{ hello }</code> |
| <code>d(oli)g</code> | <code>{ dog,dig }</code> |
| <code>moo*</code> | <code>{ mo,moo,mooo,... }</code> |
| <code>(moo)*</code> | <code>{ o,moo,moomoo,moomoomoo,... }</code> |
| <code>a(b a)*a</code> | <code>{ aa,aaa,aba,aaaa,aaba,abaa,... }</code> |
- The syntax described so far is entirely sufficient to write any regular expression.
- But, it is also handy to have a few helper operations built on top of the basic syntax:

Helper Operations

$s?$ indicates that s is optional.

$s?$ can be written as $(s|q)$

$s+$ indicates that s is repeated one or more times.

$s+$ can be written as ss^*

$[a-z]$ indicates any character in that range.

$[a-z]$ can be written as $(a|b|\dots|z)$

$[^x]$ indicates any character except one.

$[^x]$ can be written as $\Sigma -$

Regular expressions...

- Regular expressions also obey several algebraic properties, which make it possible to re-arrange them as needed for efficiency or clarity:
 - Associativity: $a|(b|c) = (a|b)|c$
 - Commutativity: $a|b = b|a$
 - Distribution: $a(b|c) = ab|ac$
 - Idempotency: $a^{**} = a^{*}$

Regular Expressions...

- Using regular expressions, we can precisely state what is permitted in a given token.
- Suppose we have a hypothetical programming language with the following informal definitions and regular expressions.
- For each token type, we show examples of strings that match (and do not match) the regular expression.
 - Informal definition: ***An identifier is a sequence of capital letters and numbers, but a number must not come first.***
 - Regular expression: `[A-Z]+([A-Z]|[0-9])*`
 - Matches strings: PRINT
MODE5
 - Does not match: hello
4YOU

Regular Expressions...

- Informal definition: *A number is a sequence of digits with an optional decimal point. For clarity, the decimal point must have digits on both left and right sides.*
 - Regular expression: $[0-9]+(\.[0-9]+)?$
 - Matches strings: 123
3.14
 - Does not match: .15
30.

Regular Expressions...

- Informal definition: *A comment is any text (except a right angle bracket) surrounded by angle brackets.*
 - Regular expression: `<[*> ]*>`
 - Matches strings: `<tricky part>`
`<<<< look left>`
 - Does not match: `< this is an <illegal> comment>`

Regular Expressions: Basic Symbols

- Basis symbols:
 - ε is a regular expression denoting language $\{\varepsilon\}$
 - $a \in \Sigma$ is a regular expression denoting $\{a\}$
- If r and s are regular expressions denoting languages $L(r)$ and $M(s)$ respectively, then
 - $r \mid s$ is a regular expression denoting $L(r) \cup M(s)$
 - rs is a regular expression denoting $L(r)M(s)$
 - r^* is a regular expression denoting $L(r)^*$
 - (r) is a regular expression denoting $L(r)$
- A language defined by a regular expression is called a **Regular Set**

Regular Expressions and their Derivation

Regex	Derivation rules	Informal description
a		The one-letter string a . No derivation rule, as it is already a string.
ϵ	$\epsilon \Rightarrow$	The empty string.
$s t$	$s t \Rightarrow s$ $s t \Rightarrow t$	Either s or t . Note that this allows multiple different derivations.
st	$st \Rightarrow s't'$, if $s \Rightarrow s'$ and $t \Rightarrow t'$	Something derived from s followed by something derived from t .
s^*	$s^* \Rightarrow$ $s^* \Rightarrow s(s^*)$	A concatenation of any number (including 0) of strings derived from s . The number depends on how many times the second rule is used.

Regular Expressions: Notational Short-hands

- We frequently use the following shorthands:

$$r^+ = rr^*$$

$$r? = r \mid \varepsilon$$

$$[\mathbf{a-z}] = \mathbf{a} \mid \mathbf{b} \mid \mathbf{c} \mid \dots \mid \mathbf{z}$$

- For example:

$$\mathbf{digit} \rightarrow [\mathbf{0-9}]$$

$$\mathbf{num} \rightarrow \mathbf{digit}^+ (.\mathbf{digit}^+)? (\mathbf{E} (+|-)? \mathbf{digit}^+)?$$

Regular Expressions: Regular Definitions

- Naming convention for regular expressions:

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

- where r_i is a regular expression over $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$
 - Each d_j in r_i is textually substituted in r_i

Regular Definitions

- Example:

letter $\rightarrow$ **A** | **B** | ... | **Z** | **a** | **b** | ... | **z**

digit $\rightarrow$ **0** | **1** | ... | **9**

id $\rightarrow$ **letter** (**letter** | **digit**)^{*}

Lecture Outlines | *Progress*

- Introduction
- What is a Language?
- Introduction to Regular Expressions (RegExps)
- Regular Definitions and Notations
- **Derivations, Parenthesis and Precedence Rules**
- Use of RegExps in a Typical Programming Language
- RegExp and NFA

Derivation of Strings Using Rules

- To find $L(s)$ for a given regular expressions, we use **derivation**.
- **What is Derivation?**
 - Rules that rewrite a regular expression into a string of letters
 - These rules allow a single regular expression to be rewritten into several different strings.
 - $L(s)$ is the set of strings that s can be rewritten to using these rules.
 $L(s)$ is often an infinite set,
 - However, each string in the set is **finite** and can be obtained by a finite number of derivation steps.

Parentheses and Precedence Rules

- When we use the symbols above to construct composite regular expressions such as a/ab^* , it is not a priori clear how the different subexpressions are grouped.
 - Sometimes parentheses are used to make the grouping of symbols explicit such as in $(a/(ab))^*$.
 - Additionally, we use precedence rules, similar to the algebraic convention that multiplication binds stronger than additions, so $3+4 \times 5$ is equivalent to $3+(4 \times 5)$ and not $(3+4) \times 5$.
- For regular expressions, we use the following conventions:
 - $*$ binds tighter than **concatenation**,
 - Which binds tighter than **alternative** ($/$).
 - The example a/ab^* from above is, hence, equivalent to $a/(a(b^*))$

Some Algebraic Properties of Regular Expressions

$$(r|s)|t = r|s|t = r|(s|t)$$

| is associative.

$$s|t = t|s$$

| is commutative.

$$s|s = s$$

| is idempotent.

$$s? = s|\varepsilon$$

by definition.

$$(rs)t = rst = r(st)$$

concatenation is associative.

$$s\varepsilon = s = \varepsilon s$$

ε is a neutral element for concatenation.

$$r(s|t) = rs|rt$$

concatenation distributes over |.

$$(r|s)t = rt|st$$

concatenation distributes over |.

$$(s^*)^* = s^*$$

* is idempotent.

$$s^*s^* = s^*$$

0 or more twice is still 0 or more.

$$ss^* = s^+ = s^*s$$

by definition.

$$(s^+)^+ = s^+$$

+ is idempotent.

Describe the following as RegExps

- $\{0,1,2\}$ ***$R = 0/1/2$***
- $\{\wedge, ab\}$ ***$R = \wedge ab$***
- $\{abb, a, b, bba\}$ ***$R = abb/a/b/bba$***
- $\{\wedge, 0, 00, 000, \dots\}$ ***$R = 0^*$***
- $\{1, 11, 111, 1111, \dots\}$ ***$R = 1^+$***

Lecture Outlines | *Progress*

- Introduction
- What is a Language?
- Introduction to Regular Expressions (RegExps)
- Regular Definitions and Notations
- Derivations, Parenthesis and Precedence Rules
- **Use of RegExps in a Typical Programming Language**
- RegExp and NFA

Design a Regular Expression for the following Languages over {a,b}.

- Language accepting strings of length **exactly 2**
 - $L = \{aa, bb, ab, ba\} \rightarrow aa+bb+ab+ba$
 - $L = (a+b)(a+b)$
- Language accepting strings of length **at least 2**
 - $L = \{aa, ab, ba, bb, aaa, \dots\}$
 - $L = (a+b)(a+b)(a+b)^*$
- Language accepting strings of length **at most 2**
 - $L = \{\epsilon, a, b, aa, ab, \dots\}$
 - $L = (\epsilon+a+b)(\epsilon+a+b)$

Examples of How RegExps is used to Describe typical programming language elements

- **Keywords:**

- A keyword like **if** is described by a regular expression that looks exactly like that keyword, e.g., the regular expression **if** (which is the concatenation of the two regular expressions **i** and **f**).

- **Variable names:**

- In the programming language C, a variable name consists of letters, digits and the underscore symbol and it must begin with a letter or underscore.
 - This can be described by the RegExps **[a-zA-Z_][a-zA-Z_0-9]***

Examples of How RegExps is used to Describe typical Programming language elements

■ Integers

- An integer constant is an optional sign followed by a non-empty sequence of digits: **[+-]?[0-9]⁺**
- In some languages, a signed constant is not a single token, but a concatenation of two tokens: the sign and an unsigned number constant.
 - This will usually allow whitespace between the sign and the number, which is not possible with the above.

Examples of How RegExps is used to Describe typical programming language elements.

- **String constants**

- A string constant starts with a quotation mark followed by a sequence of symbols and finally another quotation mark.

`"([a-zA-Z0-9]|\[a-zA-Z])"`

Lecture Outlines | *Progress*

- Introduction
- What is a Language?
- Introduction to Regular Expressions (RegExps)
- Regular Definitions and Notations
- Derivations, Parenthesis and Precedence Rules
- Use of RegExps in a Typical Programming Language
- **RegExp and NFA**

Finite Automata

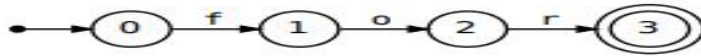
- A finite automaton (FA) is an abstract machine that can be used to represent certain forms of computation.
- Graphically, an FA consists of a number of states (represented by numbered circles) and a number of edges (represented by labeled arrows) between those states.
- Each edge is labeled with one or more symbols drawn from an alphabet Σ
- The machine begins in a start state S_0 . For each input symbol presented to the FA, it moves to the state indicated by the edge with the same label as the input symbol

Finite Automata...

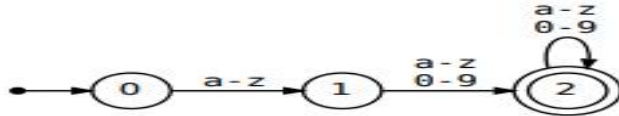
- Some states of the FA are known as accepting states and are indicated by a double circle.
- If the FA is in an accepting state after all input is consumed, then we say that the FA accepts the input.
- We say that the FA rejects the input string if it ends in a non-accepting state, or if there is no edge corresponding to the current input symbol.

Finite Automata...

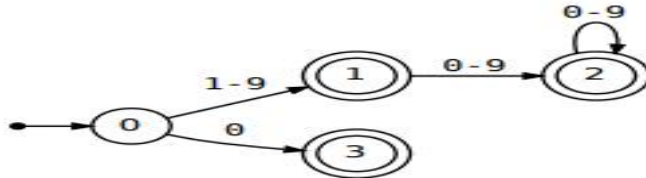
- Every RE can be written as an FA, and vice versa. For a simple regular expression, one can construct an FA by hand. For example, here is an FA for the keyword **for**:



Here is an FA for identifiers of the form $[a-z][a-zA-Z0-9]^+$



And here is an FA for numbers of the form $([1-9][0-9]^*)|0$



Deterministic Finite Automata

- Each of these three examples is a deterministic finite automaton (DFA).
- A DFA is a special case of an FA where every state has no more than one outgoing edge for a given symbol.
- Put another way, a DFA has no ambiguity: for every combination of state and input symbol, there is exactly one choice of what to do next.
- Because of this property, a DFA is very easy to implement in software or hardware. One integer (c) is needed to keep track of the current state.

Deterministic Finite Automata...

- The transitions between states are represented by a matrix ($M[s, i]$) which encodes the next state, given the current state and input symbol.
- (If the transition is not allowed, we mark it with E to indicate an error.)
- For each symbol, we compute $c = M[s, i]$ until all the input is consumed, or an error state is reached.

NonDeterministic Finite Automata (NFA)

- To transform **Regular Expressions** into efficient programs, **NFA** are used as a stepping stone.
 - By their nondeterministic nature, these are not quite as close to “real machines” and can be transformed into **Deterministic Finite Automata (DFA)**, which are easily and efficiently executable on normal hardware.
- **A finite automaton** is, in the abstract sense, a machine that has
 - a **finite number of states** and
 - a **finite number of transitions** between pairs of states

Nondeterministic Finite Automata (NFA)

- A finite automaton can be used to decide if an input string is a member in some particular set of strings.
- To do this, we select one of the states of the automaton as the **starting state**.
 - We start in this state, and in each step we can do one of the following:
 - Follow an epsilon ϵ transition to another state, or
 - Read a character from the input and follow a transition labelled by that character.

Nondeterministic Finite Automata (NFA)

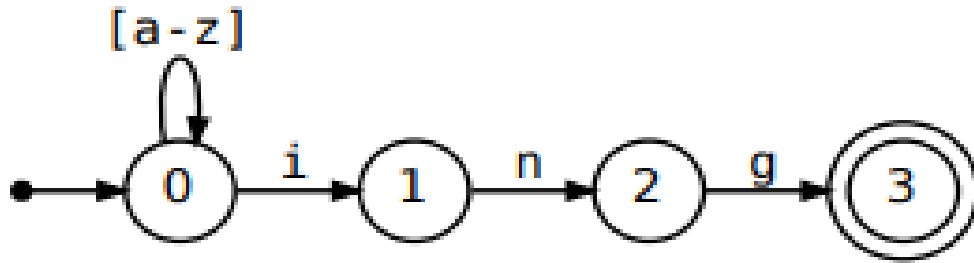
- At each step, we may have a choice of several actions:
 - We can choose between either an epsilon transition or a transition on an alphabet character, and if there are several transitions with the same symbol, we can choose between these.
- This makes the automaton **nondeterministic**,
 - As the choice of action is not determined solely by looking at the current state and the next input character.

Nondeterministic Finite Automata (NFA)

- **Transitions:** used to reach a new state from the current state,
 - either follow an ϵ transition (no char of input is consumed) or
 - consume the current char c of the input and follow a transition labelled with c from the current state (if such transition exists),
- **Membership:** if there is a possibility to reach an accepting state after all input characters were consumed then input $\in L$.
 - **Possibility** refers to choices on what transaction to follow

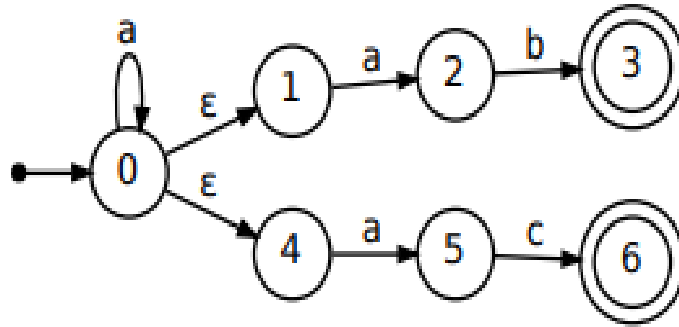
Nondeterministic Finite Automata

- The alternative to a DFA is a nondeterministic finite automaton (NFA).
- An NFA is a perfectly valid FA, but it has an ambiguity that makes it somewhat more difficult to work with
- Consider the regular expression $[a-z]^*ing$, which represents all lowercase words ending in the suffix *ing*. It can be represented with the following automaton



Nondeterministic Finite Automata

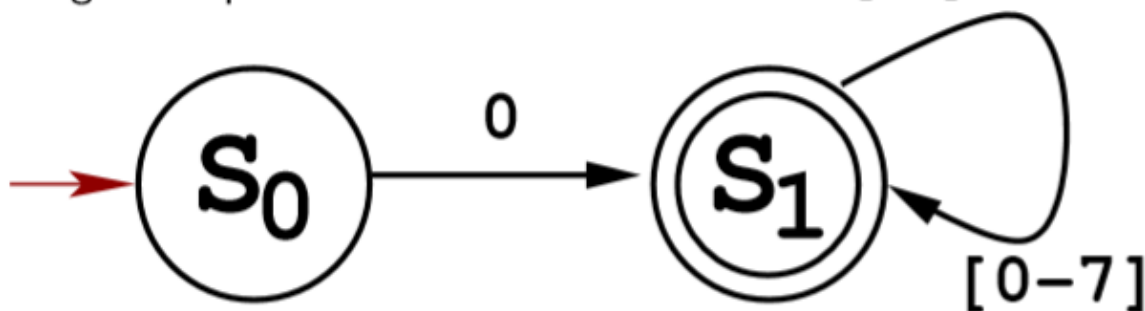
- An NFA can also have an ϵ (epsilon) transition, which represents the empty string. This transition can be taken without consuming any input symbols at all. For example, we could represent the regular expression $a^*(ab|ac)$ with this NFA:



- This particular NFA presents a variety of ambiguous choices. From state zero, it could consume a and stay in state zero. Or, it could take an ϵ to state one or state four, and then consume an a either way.

NFA for Octal Number

Regular Expression for an Octal Number: $0 [0-7]^*$



- Start in state S_0
 - **IF** input is 0 go to state S_1
 - **OTHERWISE** analysis fails!
- In state S_1 :
 - **IF** input in $0 \dots 7$ stay there
 - **OTHERWISE** analysis fails!
 - **If end of input reached then success: an octal number was identified!**

Converting a Regular Expressions to NFA

- NFA are contrasted **compositionally** from a regular expression;
 - Each subexpression construct an NFA fragment and then combine these fragments into bigger fragments.
 - A fragment is not a complete NFA, so we complete the construction by adding the necessary components to make a complete **NFA**.

Converting a Regular Expressions to NFA

Regular expression

NFA fragment

a



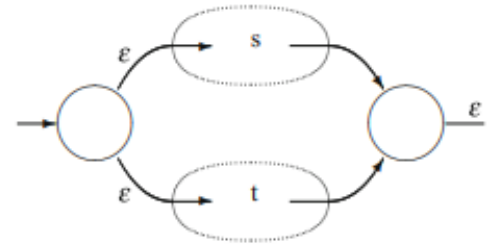
ϵ



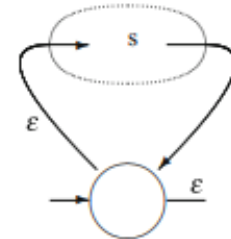
st



s|t



s^*



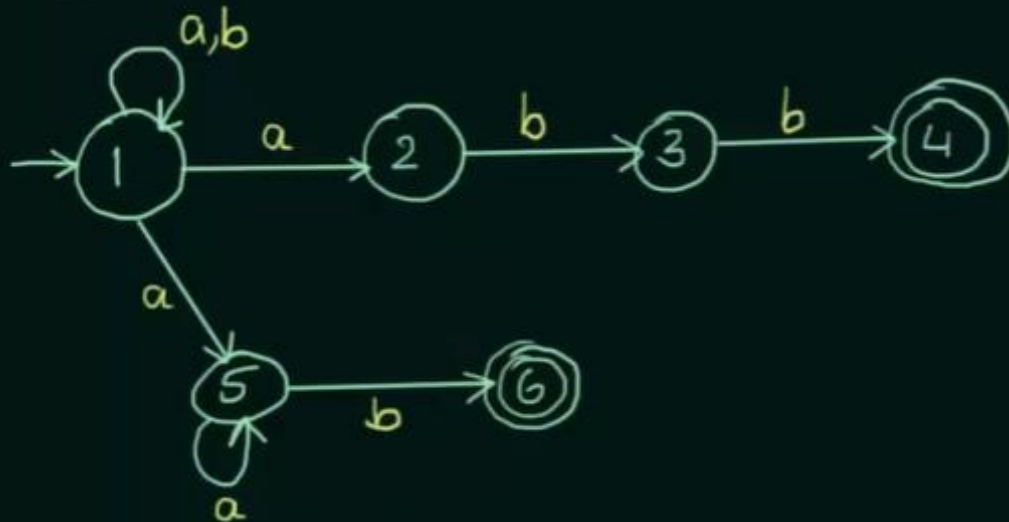
Examples

- Convert the Following Regular Expressions to its NFA

$(a|b)^*(abb|a^+b)$

$10+(0+11)0^*1$

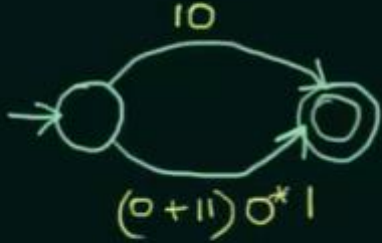
$(a|b)^* (abb|a^+b)$ +



$$a^+ = \{a, aa, aaa, \dots\}$$

$$a^* = \{\epsilon, a, aa, \dots\}$$

$$10 + (0 + 11) 0^* 1$$



Source: NESO Academy (YouTube Channel)

